

An Efficient Approach to Correct the Signals and Generate High-Resolution Quadrature Pulses for Magnetic Encoders

Hung Van Hoang and Jae Wook Jeon, *Member, IEEE*

Abstract—A magnetic encoder (ME) is a kind of sinusoidal encoder using magnetic effects that is currently utilized in many industrial control systems because it has many advantageous characteristics: low cost, simple structure, works in harsh environments, high reliability, and so on. The signals generated by an ME are always disturbed by noises; therefore, these signals are not ideal. The challenge is to achieve the highest resolution and to get the maximum operating speed as well as to use the most cost-effective hardware. To solve this problem, this paper humbly proposes an effective approach, which contains two parts: The main part is “compensating the noisy signals of MEs” by applying a new proposed method called the advanced adaptive digital phase-locked loop (AADPLL), while the second is a pulse interpolator which generates high-resolution quadrature pulses. The AADPLL algorithm provides a robust filtering characteristic to eliminate the noises and improve the accuracy of the ME’s input signals. It also takes advantage of tracking high-speed input signals without time lag, unlike the traditional filters. Additionally, the computation burden is significantly reduced in this algorithm to allow it to be easily implemented in a low-cost processor. The pulse interpolator is based on an existing idea that extracts high-order sinusoids from the original ME signals. However, a new scheme is presented to achieve higher resolution per period with smaller noises affecting the output pulses. Both parts are mainly implemented in a unique hardware platform using a low-cost digital signal processor, such as the TMS320F2812, combined with a small-size field-programmable gate array. This method has already been applied to control a linear motor without using an expensive optical linear encoder. Practical results are provided to demonstrate the effectiveness of the proposed method.

Index Terms—Adaptive digital phase-locked loop (PLL), interpolator, magnetic encoders (MEs), phase detection, pulse correction, pulse generator, sinusoidal encoders.

I. INTRODUCTION

HIGH-PRECISION control is currently, and will continue to be, an essential requirement for many industrial applications. In addition to a precise mechanical system and an effective control algorithm, the stability and accuracy of the

feedback sensors are of paramount importance for achieving excellent performance of closed-loop control systems. Currently, traditional position transducers that are based on resolvers, tachometers, and optical encoders are being frequently used in these applications. In recent times, magnetic sensors have been increasingly applied instead of the traditional ones, particularly the magnetic encoder (ME) because it offers many advantages compared with other sensors. MEs can work in harsh environments, such as oil, dirt, dust, fluctuating temperature, vibration of machine, and so on. Due to its simple structure, the ME can be manufactured as a miniature type that is suitable for controlling small systems, such as printers. ME is a cost-effective solution, and it is capable of assuring the resolution and accuracy performance of feedback information. The ME is a common type of incremental encoder that typically supports two output channels (channels A and B) that are dephased by 90° from each other. Generally, these outputs are sinusoidal signals in quadrature for easily getting higher resolution by applying a hardware or software interpolation method. However, they are always corrupted by noise and by various sensing errors: random noise, variation of ambient temperature, mechanical errors, operating environment of sensors, and so on. This causes undesired output signals that contain dc offsets, amplitude deviation, waveform distortion, and non-90° phase shifting between the sinusoidal waves. It has been a challenge to create an efficient method to compensate noises for ME signals and then to extract the high-resolution position from these signals.

Many approaches of interpolation using signal-processing techniques have been proposed in both hardware and software. A series of techniques based on the optimization theory for correcting the nonideal signals are shown in [1]–[4]. The first introduction was presented by Heydemann [1]; his least square fitting method can efficiently eliminate the signal’s errors; however, Heydemann’s method is only used as an offline compensator. Balemi [2] has proposed an online method that employs residual gradient search, but it does not show a good performance in low-frequency domains and with noisy signals. The authors’ work in [3] has solved the problem of Balemi by the following: 1) switching the suitable sampling time of two frequency domains (high and low) and 2) redesigning the system with more filters and the antiphase shifting between sinusoidal signals; however, it does not perform well in the overlapping area between low and high frequencies and during the “resting period” (no movement or at rest). The modified

Manuscript received October 23, 2009; revised April 15, 2010; accepted October 18, 2010. Date of publication November 18, 2010; date of current version July 13, 2011. This work was supported in part by MKE, Korea, under ITRC NIPA-2010-(C1090-1021-0008) and in part by the Priority Research Centers Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education, Science and Technology (2010-0020210).

The authors are with the School of Information and Communication Engineering, Sungkyunkwan University, Suwon 440-746, Korea (e-mail: hunghv@ece.skku.ac.kr; jwjeon@yurim.skku.ac.kr).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TIE.2010.2093472

recursive weighted least squares [4] is an effective approach for real-time tracking of the resolver parameters. This method can overcome the problem of the resting period that other previous optimization methods cannot. However, this method has a huge computation burden (about 2 ms), so the bandwidth of the input signals will be limited. Some lookup-table (LUT)-based approaches are discussed in [5]–[7], but these methods occupy large memory space to store the LUTs and can only adapt to particular encoders that correspond with built-up LUTs. Recently, Sarma *et al.*, in [8], have proposed a method for resolver-to-digital conversion that uses the software generation of carrier wave to synchronize with resolver outputs and, after that, determine the angle by a size-reduced LUT (only up to 45°). Tan and Zhou [9], [10] solved the problem of high resolution by calculating the high-order sinusoids of the original signals and then storing those high-order values to LUTs in advance [9]. Also, based on that idea, the neural-network approach has been mentioned in [10] with two radial basic function stages: The first stage is used to correct the imperfect signals, and the second one works as an inference machine to interpolate the quadrature pulses. Nevertheless, this method has a complicated computation and requires training time for every new encoder as well as for every new operating environment. A learning-time compensator using probabilistic method, as shown in [11], provides a fine result of obtaining smoother signals from noisy ones. However, it has some disadvantages: requiring a large number of samples that will increase the learning time, and when the operating environment is changed, noise is also changed; thus, it is necessary to repeat the “learning-step” to obtain a better result. Bunte and Beineke, in [12], analyze the error characteristics of the resolver or encoder signals as Fourier series and determine the corrected coefficients for the signals by applying the cross-correction technique. According to this method, it is necessary to calculate the integral of the squared error over a sufficient number of sinusoidal periods; this causes a drawback of the system to operate in a short distance (for example, inside one period of sinusoids). Hardware-based approaches are introduced in [13]–[16]. In there, schemes [13]–[15] based upon the pseudolinear segments of sinusoidal signals and the instrumentation note [16] report a universal magnetic circuit to form a suitable core for a family of multipole two-speed resolvers with varieties of popular speed ratios. These significantly reduce the cost by being carried out in a simple hardware circuit; however, the authors did not show how to resolve the issues of antinoise as well as their performance in low speeds, where the ME’s signals will contain a lot of ripples.

The phase-locked loop (PLL) is a promising technique in digital signal processing applications because of its noise-rejection capability and its simple implementation. A quadrature PLL proposed by Emura and Wang [17] is an effective method for correcting the resolver signals. However, they do not mention about the problems of low frequencies as well as interpolating to higher resolution. Bellini and Bifaretti [18] put forward a PLL-Kalman-filter-based method. Krah *et al.* [19] described a field-programmable gate array (FPGA)-based method with delta-sigma technology. In this method, they used a measured acceleration signal (motor current) as an input to a Luenberger observer combined with a tracking loop (PLL structure) to

improve the quality of the output signals. Another FPGA-based method is introduced in [20]. This research implemented a resolver processing unit (RPU) in the FPGA matrix (named FPGA-based RPU), which uses the angle-tracking-observer method to compute the rotor position and speed from the resolver signals, for synchronous-motor control systems. Most recently, Sayeef *et al.* [21] have presented a nice method to estimate the rotor speed and position for the sensorless interior permanent-magnet synchronous motor driver. The problem of low frequencies is solved in this approach by combining a high-frequency signal injection at low speeds with a sliding observer at medium to high speeds; the changeover between these two algorithms is performed using a weighting function which takes place when the driver is operating at middle speeds. However, these approaches, as a whole, do not address the reduction of the phase shifts between two ME signals or the time lag caused by low-pass filters (LPFs) and so on. In summary, some of the aforementioned approaches require much computation or memory to store the LUTs, while the others have limitation in the bandwidth of input signals or do not meet the high-resolution requirement. The remaining references introduce some methods which relate to the authors’ method or some books that this paper refers to.

This paper proposes a reliable approach to process the noisy signals and to interpolate high-resolution quadrature pulses for MEs, in particular, and sinusoidal encoders, in general. It will be discussed in more details in the subsequent sections.

This paper is organized as follows. In Section II, we first review some related works and briefly describe the proposed approach. The advanced adaptive digital PLL (AADPLL) is explained in detail in Section III, which presents a new algorithm to efficiently compensate for the erroneous factors of the ME signals. A new scheme that interpolates the aforementioned corrected signals to higher pulses is introduced in Section IV. Section V shows the simulation and experimental results for verification of the proposed approach. Section VI concludes this paper.

II. RELATED WORKS AND OVERVIEW OF THE PROPOSED APPROACH

The raw signals that are returned from MEs are normally a pair of sine/cosine waveforms with the addition of internal and external noises: phase shifts, dc offsets, different amplitudes, and waveform distortions. They can be expressed as follows:

$$\begin{cases} u_{\sin} = A_s \sin(\theta) + B_s + \eta_1(t) \\ u_{\cos} = A_c \cos(\theta + \delta) + B_c + \eta_2(t) \end{cases} \quad (1)$$

where A_s and A_c are the magnitudes of the sinusoids, B_s and B_c are the dc offsets, δ is the phase shift between $u_{\sin}$ and $u_{\cos}$, and $\eta_1(t)$ and $\eta_2(t)$ are random noises of the environment.

A. Related Works

In practice, noises are unpredictable and vary by operating conditions and aging. They remarkably affect the output signals of the sensors, particularly in very low operating frequencies

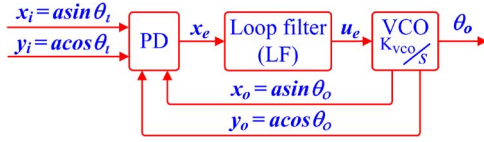


Fig. 1. Structure of traditional quadrature PLL.

of the actuator. There are many approaches that have been formerly proposed. As we verified, the PLL method is always the best one that gives a rapid computation and efficiently eliminates the noises from the sensor signals. The PLL's basic structure contains three main parts: the phase detector, the loop filter, and the voltage-controlled oscillator (VCO).

Emura and Wang, in [17], introduced a PLL-based compensator that is implemented in a combination of software and hardware, as shown in Fig. 1. The author proposes a modified PLL-type, called quadrature PLL, with a pair of sine and cosine waves as inputs, instead of the ordinary PLL structure that has only one input signal. Assuming that the amplitudes of the sine/cosine waveforms are the same, the phase detector can be shown as follows:

$$\text{In : } \begin{cases} x_i = a \sin \theta_i \\ y_i = a \cos \theta_i \end{cases} \quad \text{Out : } \begin{cases} x_o = b \sin \theta_o \\ y_o = b \cos \theta_o \end{cases} \quad (2)$$

$$\Rightarrow x_e = x_o y_i - x_i y_o = ab \sin(\theta_i - \theta_o) \approx K_{PD} \cdot e$$

$$\text{with } K_{PD} = ab \quad e = \theta_i - \theta_o. \quad (3)$$

The VCO was also built with an analog circuit using two ROMs and two D/A converters. This generates the output sine/cosine waveforms corresponding to the value of the voltage u_e that is returned from the loop filter (see Fig. 1). The provided results prove the effectiveness of this method. However, it still does not overcome the problems of bandwidth of the input signals or the error of the quadrature input phase. In reality, the phase difference between the sine and cosine of the input signal is not always 90° because the noises cause a phase shift between them, which causes wrong calculation of the traditional PLL model. Assuming that the phase shift between the two inputs is δ , we can describe them as

$$\text{In : } \begin{cases} x_i = a \sin \left(\theta_i - \frac{\delta}{2} \right) \\ y_i = a \cos \left(\theta_i + \frac{\delta}{2} \right) \end{cases} \quad \text{Out : } \begin{cases} x_o = b \sin \theta_o \\ y_o = b \cos \theta_o \end{cases} \quad (4)$$

$$\Rightarrow x_e \approx K_{PD} \cdot e + R_{PD},$$

$$\text{with } K_{PD} = ab \cos \frac{\delta}{2} \quad R_{PD} = ab \sin \frac{\delta}{2} \cos e. \quad (5)$$

From (5), it is clearly realized that the phase error changes with the variable factor R_{PD} ; this will cause an error in the traditional PLL model. Generally, the raw signals from the sensors are always processed by using LPFs, and the time lag of the LPFs also makes the same effect on this PLL model. Moreover, according to the traditional PLL structure, we cannot have both advantages at the same time: a wide bandwidth of operation and good filtering characteristics. By increasing the PLL's bandwidth, noises will be increased at the PLL's outputs; therefore, the PLL easily comes to out-of-lock state.

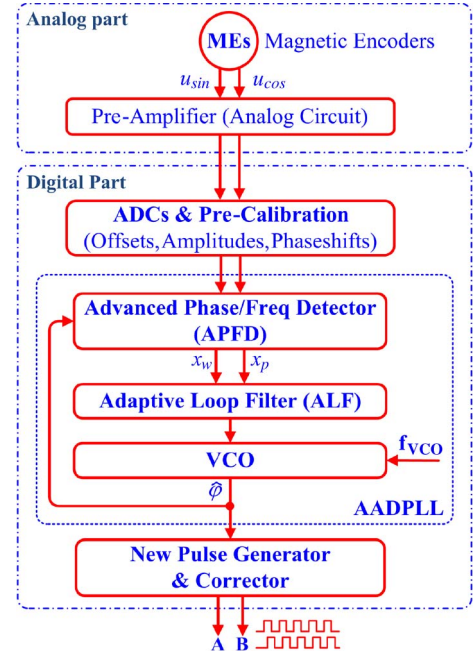


Fig. 2. General block diagram of the proposed approach.

The previous step has only compensated for the raw signals from the noises. The output frequency is the same as the original one, which means that the position resolution was not upgraded. To get a higher resolution, Tan and Zhou proposed a nice idea in their paper [9]. In one period of the original input signals $[\sin \theta_i, \cos \theta_i]$, they extracted the high-order waves $[\sin(n\theta_i), \cos(n\theta_i)]$ (where n is known as the interpolation factor) according to the LUT method. Afterward, these high-order waves were converted to binary pulses. Therefore, the resolution of the encoder is interpolated to n -enhanced position by this scheme. However, their approach did not take care about the noises affecting the magnitude of the sine/cosine input signals. This will cause a dither (or multitransition) phenomena in the output pulses.

This paper introduces a new scheme with improvements of the PLL to meet the requirements of bandwidth and achieve capability to filter noises. On the other hand, this also solves the dither problem of the output pulses by providing an upgraded method to create higher resolution of the ME signals, in particular, and the sinusoidal signals, in general. They will be presented in the next sections.

B. Proposed Approach

An overview of the proposed method is shown in Fig. 2. In this figure, the ME signals u_{sin} and u_{cos} are passed through an analog circuit that is named as the preamplifier. This circuit operates as a differential amplifier to reject the common-mode noises and to adjust the magnitude of the ME signals to satisfy the full range of the analog-to-digital converter (ADC); it also acts as an analog LPF to slightly reduce the high-frequency noises of these signals. Next, these signals are converted to digital values $u_{sin \text{ ADC}}, u_{cos \text{ ADC}}$ by two 12-b ADCs that are integrated inside the digital signal processor (DSP). These values are in the range $[0; 2^{12} - 1] = [0; 4095]$. The block

named “precalibration” is designed to eliminate the differential amplitudes and phase shifts between the two signals, and it then normalizes them to $[-1; 1]$ to be in accordance with the input range of the next block. After balancing the amplitudes of the digital input signals u_{sN}, u_{cN} (see more details in [22]), there still remains the phase shift δ between them because of mechanical errors. Without loss of generality, ignoring noises, u_{sN}, u_{cN} can be represented as

$$\begin{cases} u_{sN} = A_N \sin(\theta) \\ u_{cN} = A_N \cos(\theta + \delta). \end{cases} \quad (6)$$

The phase shift δ can be removed by applying an addition/subtraction operation of (6)

$$\begin{cases} u_{s1} = u_{cN} + u_{sN} = A_1 \sin \varphi \\ u_{c1} = u_{cN} - u_{sN} = A_2 \cos \varphi \end{cases} \quad (7)$$

where $\varphi = \theta + \theta_0$, $A_1 = 2A_N \cos \theta_0$, $A_2 = 2A_N \sin \theta_0$, and $\theta_0 = \frac{\pi + 2\delta}{4}$.

In reality, the variation of δ is so small that it is assumed to be constant. From (7), A_1 and A_2 are also considered as constants; therefore, it is easy to find two multipliers k_1, k_2 satisfying $k_1 \cdot A_1 \approx k_2 \cdot A_2 = A$. We can get two better signals u_s, u_c , which are approximately considered to be equal in amplitudes and to be dephased by a fixed angle $(\pi + 2\delta)/4$ with the original ones u_{sN}, u_{cN}

$$u_s = A \sin \varphi \quad u_c = A \cos \varphi. \quad (8)$$

However, after they are processed by the precalibration block, the sine/cosine signals still contain noises, small offsets, and deviation in amplitudes because the precalibration block works only as an offline calibrator, while these parameters are varied by time. The proposed AADPLL can adaptively track and effectively eliminate noises from the ME signals in real time after completely precalibrating. Two important improvements of the phase detector and loop filter will decide the effectiveness of this algorithm. In most cases, when we use the fixed cutoff frequency w_c of the LPFs to improve the quality of the input signals, it will cause significant time-lag results. This time lag will be increased when the frequency of the inputs comes near the cutoff frequency of the LPFs, and it then makes the PLL lose its lock-in state. For this reason, the advanced phase/frequency detector (APFD) is designed to replace the traditional one. It has three advantages: First, w_c can be dynamically varied according to the frequency of the encoder signals to increase the bandwidth of PLL and to effectively remove noises from the inputs in the very low frequency domain; second, APFD has the ability to compensate for the time lag of the affected PLL outputs by using the LPFs; and third, the APFD is upgraded by adding a frequency-aided detector (FAD) to reduce the acquisition time of the loop. Furthermore, the AADPLL has an adaptive loop filter (ALF) that can adaptively change the parameters of the overall loop; therefore, the acquisition time and the filtering characteristics are affected again. With these new features, the AADPLL can provide a wider operating bandwidth, shorter acquisition time, and better filtering characteristics than the conventional ones. Its structure is described in more details in Section III.

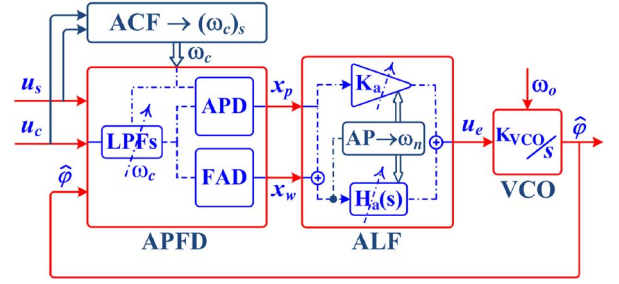


Fig. 3. Overall structure of AADPLL.

Normally, the standard output of incremental encoders used in industrial applications is a pair of quadrature pulses. Therefore, it is necessary to design an interpolator for converting sinusoids to quadrature pulses and increasing the resolution of sinusoidal encoders. In this paper, we introduce a new scheme which we called “new-pulse generator and corrector,” as shown in Fig. 2. The original idea of this converter has come from a previous paper by Tan and Zhou [9], which derives high-order sinusoids from the original ME outputs in one period and changes them to binary pulses using LUTs. In this paper, we decrease the order of extracted high-order sinusoids in order to reduce the noises of the output pulses by using an FPGA combined with the DSP hardware platform. One part of this scheme will be implemented in the FPGA to obtain eight times bigger gain in the position achieved from the aforementioned high-order sinusoids. More details for this converter are presented in Section IV. In summary, compared with the previous approaches by the same authors [22], this work is enhanced by upgrading the design of the AADPLL and the algorithm of the pulse generator as well. Therefore, we can achieve much better results than that of the previous one.

III. AADPLL

As mentioned earlier, the proposed AADPLL is composed of three main parts: APFD, ALF, and VCO (see Fig. 2). Fig. 3 shows an overall view of the AADPLL. In this model, the $\omega_{-3\text{ dB}}$ frequencies of LPFs used in APFD (shown as ω_c) will be adapted corresponding to the input-signal frequencies by “adjusting the cutoff frequency” (ACF) block, and the adjusting parameter (AP) block will adaptively adjust the natural frequency ω_n of the main loop. In addition, this will make a suitable change for the parameters of the ALF part. This will all be detailed in the next sections.

A. APFD

1) *APD*: The phase detector provides an average value that is proportional to the phase error between the input phase φ and the output phase $\hat{\varphi}$ of the PLL. This phase error is expressed by (3) in the conventional PLL. However, it is no longer correct when using the LPFs to reduce random noises because of the time-lag problem that is caused by the LPFs. To overcome this problem, this paper proposes an advanced phase detector (APD) that is added to two first-order LPFs and two second-order LPFs, as shown in Fig. 4. The first-order ones are used as average filters to remove high-frequency components that

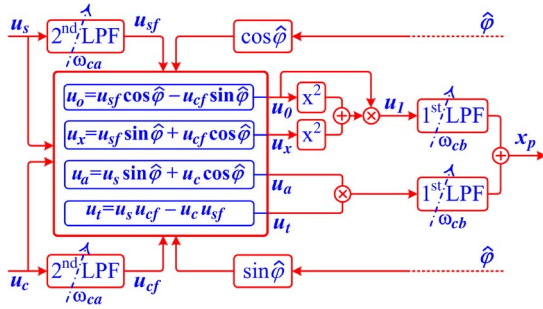


Fig. 4. Implementation of APD.

are higher than the bandwidth of the input signals for the intermediate signals u_1 , u_a , and u_t (see Fig. 4), and their phase lags do not make a significant effect on the convergence of the AADPLL. The second-order ones remove noises for the input signals; nonetheless, they will cause a phase lag β to output u_{sf} , u_{cf}

$$u_{sf} = A \sin(\varphi - \beta) \quad u_{cf} = A \cos(\varphi - \beta). \quad (9)$$

The objective of this APD is to achieve a reasonable phase-error equation that gives the best description for the signal error in the case of using LPFs. From Fig. 3, we can extract the phase error x_p equation as

$$x_p = u_1 + u_a u_t = u_0 (u_0^2 + u_x^2) + u_a u_t. \quad (10)$$

In (10), u_0 , u_x , u_a , u_t are expressed by the phase difference $e_p = \varphi - \hat{\varphi}$ as follows:

$$\begin{cases} u_0 = u_{sf} \cos \hat{\varphi} - u_{cf} \sin \hat{\varphi} = A \sin(e_p - \beta) \\ u_x = u_{sf} \sin \hat{\varphi} + u_{cf} \cos \hat{\varphi} = A \cos(e_p - \beta) \\ u_a = u_s \sin \hat{\varphi} + u_c \cos \hat{\varphi} = A \cos e_p \\ u_t = u_s u_{cf} - u_c u_{sf} = A^2 \sin \beta. \end{cases} \quad (11)$$

Substituting (11) into (10) yields

$$x_p = A^3 \cos \beta \sin e_p \approx K_{APD} \cdot e_p$$

$$\text{where } K_{APD} = A^3 \cos \beta. \quad (12)$$

Equation (12) shows that the proposed phase detector provides a highly linear relationship between the output x_p and the phase difference e_p between the input phase φ and the estimating phase $\hat{\varphi}$. Normally, β is treated as a constant at each input frequency and kept to a small enough value by the ACF block; therefore, K_{APD} is also considered as a constant at each input frequency.

2) *FAD*: Acquisition time is an important factor to be considered in designing the PLL. When a loop starts out in an unlocked condition, it must be brought into a lock state. The process of bringing the loop into lock should be as fast as possible to lessen the acquisition time of the PLL. This also means that the speed of the ME signal recovery from noises will be significantly reduced. From many references and analyses, we realize that it is required to add a frequency-aided circuit to the PLL rather than by only using a phase detector, as in the previous designs. The FAD block is introduced to undertake this function, as shown in Fig. 3. When a pair of sinusoidal signals

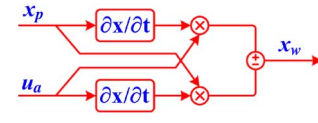


Fig. 5. Implementation of FAD.

is applied to the AADPLL, if the difference frequency $\Delta\omega$ between the input and the VCO is large, then a beat note will appear at the APD output with a dc-level near zero. Therefore, the loop will slowly come to steady state if only the phase detector is utilized in this case. However, the loop will be quickly driven into the lock-in state with the aid of the FAD, which generates a value being proportional to the difference $\Delta\omega$ and then adds it to the error output of the AADPLL. When the error is large, the AADPLL will adapt itself to reduce it and rapidly converge to the lock-in state.

The principle of the FAD originates in the relationship between angular velocity and phase angle $\omega = \partial\varphi/\partial t$, as shown in Fig. 5. Taking the derivative of the signals u_a and x_p yields the following:

$$\begin{cases} \dot{x}_p = A^3 \cos \beta \dot{e}_p \cos e_p \\ \dot{u}_a = -A \dot{e}_p \sin e_p. \end{cases} \quad (13)$$

Performing some extra trigonometric operations on the signals u_a , x_p and on their derivatives $\dot{u}_a$, $\dot{x}_p$, we can derive the frequency error equation as

$$x_w = u_a \dot{x}_p - \dot{u}_a x_p = A^4 \cos \beta \cdot \dot{e}_p. \quad (14)$$

Take notice that the phase difference $e_p = \varphi - \hat{\varphi}$ and the input/output frequencies of the loop $\omega_i = \dot{\varphi}$, $\omega_o = \dot{\hat{\varphi}}$; thus

$$\dot{e}_p = \dot{\varphi} - \dot{\hat{\varphi}} = \omega_i - \omega_o = e_w. \quad (15)$$

Combining (14) and (15), we finally obtain

$$x_w = K_{FAD} \cdot e_w \text{ with } K_{FAD} = A^4 \cos \beta. \quad (16)$$

We can see that the frequency error x_w is a linear function with the frequency difference e_w between the input and the output of the AADPLL. This error collaborates with the phase error to quickly lead the loop into the lock-in state; hence, the acquisition time of the AADPLL will be significantly reduced.

3) *Adjusting the Cutoff Frequencies of the LPFs*: In reality, the LPFs affect only the signal with a frequency that is near the cutoff frequency of the filters. With frequencies that are far from this cutoff frequency or that are in a very low domain (lower than 1 Hz), the noise-suppression ability of the LPFs is gradually decreased; thus, the output signals of LPFs are still noisy. For example, applying a second-order LPF with a cutoff frequency $f_c = 1500$ Hz, the signals in the range from 80 to 1400 Hz will be suppressed from noises that are better than the lower range. However, the signal-to-noise ratio of the LPF outputs will be decreased with respect to frequencies in the lower range (0 to 80 Hz). Consequently, adjustment of the LPFs's cutoff frequency in accordance with the input-signal frequency is necessary to get better results. In this paper, we will present a simple method to adjust the cutoff frequencies ω_{ca} and ω_{cb} of the LPFs used in the APD and as shown in

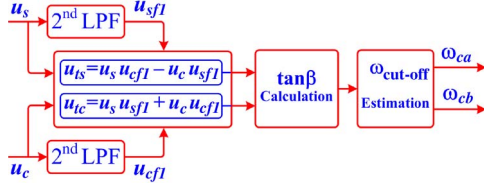


Fig. 6. Adjustment of cutoff frequency for LPFs.

Fig. 6. The basic idea comes from the relation among cutoff frequency ω_c , input frequency ω_i , and phase delay β between the input/output signals. This relationship depends on the order of the filters (more details in [32, Ch. 2]). First of all, let us discuss the method to estimate the input frequency. The transfer function of a Butterworth LPF, generally, is defined as

$$F(s) = \frac{1}{(1/\omega_c^2)s^2 + (2q/\omega_c)s + 1} \quad (17)$$

where ω_c is the -3 -dB frequency and q is the quality factor. Corresponding to each input frequency ω_i , the phase delay β is calculated as follows [replace $s = j\omega_i$ to the transfer function (17)]:

$$\tan \beta = \frac{2q\omega_c\omega_i}{\omega_c^2 - \omega_i^2}. \quad (18)$$

It is easy to see that $\tan \beta$ must be greater than zero. Transforming (18) to the quadratic form of ratio $x = \omega_i/\omega_c$ yields

$$x^2 + \frac{2q}{\tan \beta}x - 1 = 0. \quad (19)$$

Solving (19) with constraint $x \geq 0$, we get the solution

$$\frac{\omega_i}{\omega_c} = \frac{-q + \sqrt{q^2 + \tan^2 \beta}}{\tan \beta}. \quad (20)$$

In order to avoid the complicated calculation of the square-root expression, the right side of (20) should be approximated to a simpler function of $y = \tan \beta$. By using the CFTOOL in Matlab, the best fit expression is a rational equation with parabolic form for both numerator and denominator as follows:

$$\frac{\omega_i}{\omega_c} \approx \frac{p_1 y^2 + p_2 y + p_3}{y^2 + q_1 y + q_2} \quad (21)$$

where $p_1 \approx 0.9977$, $p_2 \approx 0.3932$, $p_3 \approx 0.00177$, $q_1 \approx 0.6197$, and $q_2 \approx 0.707$. This equation shows that, when ω_c is known and if $y = \tan \beta$ can be defined, then the input frequency ω_i is easily estimated. To do this, we use two other second-order LPFs with the fixed cutoff frequency $\omega_{c\beta}$ for the encoder signals u_s and u_c , as seen in Fig. 6. The outputs of the LPFs will have a phase-delayed angle β_1 similar to (9)

$$u_{sf1} = A \sin(\varphi - \beta_1) \quad u_{cf1} = A \cos(\varphi - \beta_1). \quad (22)$$

Let us take two additional calculations

$$\begin{cases} u_{ts} = u_s u_{cf1} - u_c u_{sf1} = A^2 \sin \beta_1 \\ u_{tc} = u_s u_{sf1} + u_c u_{cf1} = A^2 \cos \beta_1. \end{cases} \quad (23)$$

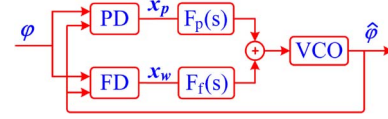


Fig. 7. A traditional PLL scheme using phase and frequency detectors.

From (23), $\tan \beta_1$ is given by a division

$$\tan \beta_1 = \frac{u_{ts}}{u_{tc}}. \quad (24)$$

After getting $\tan \beta_1$, the input frequency ω_i is easily determined by substituting $\omega_c = \omega_{c\beta}$ and $y = \tan \beta_1$ to (21). Next, we need to estimate the suitable cutoff frequencies ω_{ca}, ω_{cb} in the APD block corresponding to ω_i . This will keep the phase delay of the LPFs in an acceptable range in which the AADPLL still has good filtering characteristics. This estimation is implemented by the “ ω_{cutoff} estimation” block. It can be described as

$$\begin{cases} \omega_{ca} = K_{ca} \cdot \omega_i, & \text{with } \omega_{ca} \in [\omega_{ca \min}, \omega_{ca \max}] \\ \omega_{cb} = K_{cb} \cdot \omega_i, & \text{with } \omega_{cb} \in [\omega_{cb \min}, \omega_{cb \max}] \end{cases} \quad (25)$$

where K_{ca} and K_{cb} are constants. By experiments, we realize that if β is in the range 5° – 15° , the loop will provide a better result. Therefore, K_{ca} will be approximately in the range of [5;17] [inferred by using (20)]. On the other hand, as aforementioned, ω_{cb} is chosen to be a small enough value to remove the high-frequency components for the intermediate signals of the APD. Normally, the latter value tends to be closer to the input frequency to ensure the convergence of the AADPLL. From the experimental results, K_{cb} will also be picked out in the range of [2;5] in this application. By the aid of this additional block, we can ensure the filtering characteristics, particularly in the low input frequency domain, as well as increase the operating bandwidth of the AADPLL.

B. ALF and VCO

First, we consider a typical block diagram of the PLL with a combination of phase and frequency detectors, as shown in Fig. 7. In this model, the phase/frequency detector uses two separate loop filters: $F_p(s)$ and $F_f(s)$; however, this will cause some difficulties in designing the PLL, such as the difficulty of choosing the parameters because of the high-order problem. The second-order type is mostly preferred in implementation of the PLL and is used in many control systems because it has a simple structure and also has characteristics which are useful for control systems. Furthermore, the use of two parameters, “natural frequency ω_n ” and “damping factor ζ ,” is an advantage of this type (more in [30] and [31]). In this paper, to reduce the order of the PLL and to keep it as a second-order type, the loop filter of the frequency loop will be shared with the phase loop, as shown in Fig. 8. From previous work [22], an active lead-lag filter (ALLF) was used as a loop filter for the phase loop; in this approach, we split it into two parts: one proportional term K_a and one “type 1 transfer function” $H_a(s)$ (actually, it has the form of a first-order LPF). Moreover, the frequency loop will share $H_a(s)$ as its loop filter, while the phase loop still uses

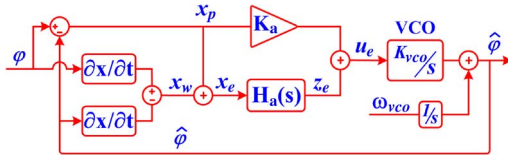


Fig. 8. Simplified block diagram of proposed AADPLL.

the ALLF as its loop filter. The ALLF transfer function can be transformed to this form

$$F(s) = K_a + \frac{K_b}{1 + s\tau_1} = K_a + H_a(s). \quad (26)$$

Generally, VCO is just an integrator. The simplified block diagram of the proposed AADPLL is shown in Fig. 8. Without loss of generality, we can assume that $K_{VCO} = 1$ and take notice that $K_{APD} \approx 1$ and $K_{FAD} \approx 1$. After the phase lock has occurred, the closed-loop transfer function of the AADPLL is given by

$$\begin{aligned} H(s) = \frac{\hat{\varphi}(s)}{\varphi(s)} &= \frac{\left(K_a + \frac{K_b}{\tau_1}\right)s + \frac{K_a + K_b}{\tau_1}}{s^2 + \left(K_a + \frac{K_b + 1}{\tau_1}\right)s + \frac{K_a + K_b}{\tau_1}} \\ &= \frac{\left(2\zeta\omega_n - \frac{\omega_n^2}{K_a + K_b}\right)s + \omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \end{aligned} \quad (27)$$

where $\omega_n = \sqrt{\frac{K_a + K_b}{\tau_1}}$ and $2\zeta\omega_n = K_a + \frac{K_b + 1}{\tau_1}$.

As known, ω_n is the natural frequency, and ζ is the damping factor. The frequency error of the AADPLL can be derived as

$$E_w(s) = \omega_i(s) - \omega_o(s) = \frac{\omega_i - \omega_{vco}/s}{1 + H_a(s)}. \quad (28)$$

Applying a step frequency change to the loop input $\omega_i = \omega_1/s$ and transforming (28) to time domain, we have

$$e_w(t) = \omega_1(t) - \omega_o(t) = \frac{\omega_1 - \omega_{vco}}{K_b + 1} \left(1 + K_b e^{-t \frac{K_b + 1}{\tau_1}}\right). \quad (29)$$

We see that, when the time constant $T = (K_b + 1)/\tau_1$ is large, the loop acquisition time will be reduced. That means that K_b should be chosen as large as possible and/or τ_1 should be decreased as small as possible. However, in practice, these parameters are limited by other constraints. The object is to define a set of suitable parameters K_a , K_b , and τ_1 from ω_n and ζ to make the best properties for the AADPLL. Without loss of generality, assume that $K_b = C = \text{const}$; from (27), we infer

$$\begin{cases} \omega_n^2 = \frac{K_a + C}{\tau_1} \\ \frac{1}{\tau_1} = \frac{1}{C+1}(2\zeta\omega_n - K_a). \end{cases} \quad (30)$$

Eliminating for the variable τ_1 in (30) finally yields

$$K_a^2 - (2\zeta\omega_n - C)K_a + (C + 1)\omega_n^2 - 2C\zeta\omega_n = 0. \quad (31)$$

Equation (31) is in a quadratic form with $\Delta = (2\zeta\omega_n + C)^2 - 4(C + 1)\omega_n^2$. This equation has a solution if and only if

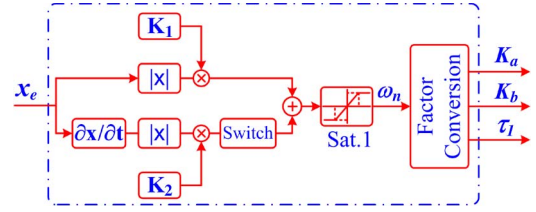


Fig. 9. AP scheme for ALF.

$\Delta \geq 0$. However, to make a simple calculation, it is reasonable to assume that (31) has a double-root (or $\Delta = 0$); therefore

$$\begin{cases} \Delta = 0 \\ K_a = \frac{2\zeta\omega_n - C}{2} \end{cases} \Rightarrow \begin{cases} \zeta = \sqrt{C+1} - \frac{C}{2\omega_n} \\ K_a = \sqrt{C+1}\omega_n - C. \end{cases} \quad (32)$$

Combining (30) and (32), we finally achieve

$$\begin{cases} K_a = \sqrt{C+1}\omega_n - C \\ \tau_1 = \sqrt{C+1}/\omega_n \\ \omega_n > C/\sqrt{C+1}. \end{cases} \quad (33)$$

As aforementioned, C is a known constant; therefore, $\sqrt{C+1}$ can be calculated in advance. The selection of a C value depends on the practical results. It will affect the damping factor ζ , K_a , and τ_1 of the system. Ordinarily, $K_b = 1$ is preferred because of its simplicity in design.

Next, (33) shows that the factors K_a and τ_1 can be defined as belonging to the natural frequency ω_n . As we know, ω_n is an important coefficient of the PLL system; it decides the stability, dynamic characteristics, filtering ability, and operating bandwidth of the PLL. Most of the previous approaches set this factor as a constant. However, a contradiction will take place by choosing ω_n in practice: When ω_n is large, the PLL's acquisition time will be small, but the noise affecting the output signals will be big (it is proportional to the value of ω_n). This totally causes undesired outputs such that we cannot get high resolution from them. To solve this problem, this paper proposes a software method to adaptively calculate the value of ω_n for the AADPLL. This mechanism is shown in Fig. 9 with the block called the APs. The basic idea came from the impact of ω_n on the PLL system: In the lock-in state, ω_n should be kept as a small enough value to maintain good filtering characteristics for the AADPLL; in the transient state (or out-of-lock state), ω_n should be set as a large value to quickly bring the AADPLL into the lock-in state. On the other hand, we realized that there is a proportional relation between ω_n and the loop error x_e (see Fig. 8). This error has also a small value in the lock-in state that becomes larger in the transient state. In particular, the variation rate of this error, which is represented by the error derivative, is clearly changed in the out-of-lock state. Based on this property, the block AP is developed to adjust ω_n in accordance with the change of the loop error x_e and its variation rate $\partial x_e / \partial t$. Then, the parameters for the loop filter are calculated. The scheme to estimate ω_n is shown in Fig. 9. It can be described as follows:

$$\omega_n = \begin{cases} K_1 |x_e|, & \text{if } x_e \frac{\partial x_e}{\partial t} < 0 \\ K_1 |x_e| + K_2 \left| \frac{\partial x_e}{\partial t} \right|, & \text{if } x_e \frac{\partial x_e}{\partial t} \geq 0 \end{cases} \quad (34)$$

where $\omega_n \in [\omega_{n \min}, \omega_{n \max}]$ and K_1 and K_2 are constants.

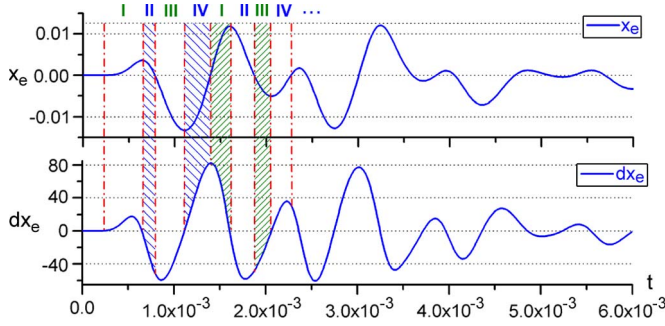


Fig. 10. Calculating scheme for natural frequency ω_n .

The addition of component $K_2|dx_e|$ in (34) is determined by the value of product $x_e dx_e$ being greater or less than zero. It is clearly explained by looking at Fig. 10. In regions I and III, the absolute value of the loop error is increasing, so ω_n needs to be large to reduce it; therefore, the $K_2|dx_e|$ component is added to the calculation of ω_n . However, $|x_e|$ has a downward tendency to be zero in regions II and IV; thus, ω_n should be reduced in this case. Hence, the $K_2|dx_e|$ component is removed from the formula of ω_n .

It can be easily seen that this condition that happens in regions I and III is $x_e dx_e \geq 0$, and the one that corresponds with regions II and IV is $x_e dx_e < 0$. Accordingly, (34) can be finally derived by basing on these statements. The amplitude of ω_n has a limited stable range in practice. It depends on the bandwidth of the input signals that we need and on the speed of the actuator that the encoder is mounted on.

In order to analyze the tracking performance of the AADPLL, the phase-error transfer function $H_e(s)$ is considered

$$H_e(s) = \frac{\varphi_e(s)}{\varphi(s)} = \frac{s \left(s + \frac{1}{\tau_1} \right)}{s^2 + 2\zeta\omega_n s + \omega_n^2}, \quad \varphi_e = \varphi - \hat{\varphi}. \quad (35)$$

A phase step input $\varphi(s) = \Delta\varphi/s$ is substituted into (35) to get

$$\varphi_e(s) = \frac{\Delta\varphi \left(s + \frac{1}{\tau_1} \right)}{s^2 + 2\zeta\omega_n s + \omega_n^2}. \quad (36)$$

Applying the final-value theorem, the steady-state error is given as

$$\lim_{t \rightarrow \infty} \varphi_e(t) = \lim_{s \rightarrow 0} s\varphi_e(s) = \lim_{s \rightarrow 0} \frac{\Delta\varphi \left(s + \frac{1}{\tau_1} \right) s}{s^2 + 2\zeta\omega_n s + \omega_n^2} = 0. \quad (37)$$

Similarly, with a step change of the frequency input $\varphi(s) = \Delta\omega/s^2$, we have

$$\varphi_v = \lim_{s \rightarrow 0} s\varphi_e(s) = \lim_{s \rightarrow 0} \frac{\Delta\omega \left(s + \frac{1}{\tau_1} \right)}{s^2 + 2\zeta\omega_n s + \omega_n^2} = \frac{\Delta\omega}{\tau_1 \omega_n^2}. \quad (38)$$

Combining (38) with (33), we infer $\varphi_v = (\Delta\omega/\tau_1\omega_n^2) = (\Delta\omega/\sqrt{C+1}\omega_n)$. Because the input frequency never agrees exactly with the VCO frequency ω_0 , it always causes a small difference error. However, if ω_n is large enough, then φ_v will approach closer to zero. On the other hand, the phase error in

this case is actually a sinusoidal form; therefore, the exact phase expression is $\sin \varphi_v = \Delta\omega/\sqrt{C+1}\omega_n$. From the condition $\sin \varphi_v \leq 1$, we have

$$\frac{\Delta\omega}{\sqrt{C+1}\omega_n} \leq 1 \Rightarrow \omega_n \geq \frac{\Delta\omega}{\sqrt{C+1}}. \quad (39)$$

Equation (39) shows that ω_n has a minimum bound. For example, in the following design with a bandwidth of 1 kHz and a VCO free-running frequency of $f_0 = 500$ Hz, by choosing $C = 1$ and $\Delta\omega_{\max} = 2\pi \times 500$ rad, we can derive $\omega_{n\min} \approx 2\pi \times 354$ rad.

C. Implementation of the AADPLL on a Fixed-Point DSP

This section presents the implementation of AADPLL to reduce the calculation burden and to avoid overflow of the fixed-point DSP. All signals are assigned as shown in Fig. 8. To realize the AADPLL on a DSP, normally, we have to change the open-loop transfer function from the Laplace domain to the discrete domain. By applying the bilinear transform with sampling time T_s to Fig. 8, we can easily get the final equations in discrete domain as follows:

$$z_{ek} = Az_{ek-1} + B(x_{ek} + x_{ek-1}) \quad (40)$$

$$\begin{aligned} \hat{\varphi}_k &= \hat{\varphi}_{k-1} + \frac{T_s}{2}(z_{ek} + z_{ek-1}) \\ &\quad + D(x_{pk} + x_{pk-1}) + E \end{aligned} \quad (41)$$

where $A = (2\tau_1 - T_s)/(2\tau_1 + T_s)$, $B = K_b T_s/(2\tau_1 + T_s)$, $D = K_a T_s/2$, and $E = \omega_0 T_s$.

However, T_s is fairly small in practice, e.g., $T_s = 10^{-5}$. Therefore, the second term in (41) $(T_s/2)(z_{ek} + z_{ek-1})$ can cause an error in calculation because of the limitation of floating numbers of the DSP families. To avoid this, we change the earlier discrete equations to a new set by putting $t_k = (T_s/2)(z_{ek} + z_{ek-1})$ to obtain

$$t_k = At_{k-1} + B'(x_{ek} + 2x_{ek-1} + x_{ek-2}) \quad (42)$$

$$\hat{\varphi}_k = \hat{\varphi}_{k-1} + t_k + D(x_{pk} + x_{pk-1}) + E. \quad (43)$$

The coefficients in (42) and (43) are defined from (33)

$$\begin{cases} A = \frac{2\sqrt{C+1}-K_w}{2\sqrt{C+1}+K_w} \\ B' = \frac{CT_s}{2} \frac{K_w}{2\sqrt{C+1}+K_w}, \\ D \approx \frac{K_w\sqrt{C+1}}{2} \\ E = \omega_0 T_s. \end{cases} \quad \text{with } K_w = \omega_n T_s \quad (44)$$

In this paper, all derivative terms are calculated by using the Taylor difference formula with third-order approximation (see [33, Ch. 5])

$$\partial x/\partial t \approx (-2x_{k-3} + 9x_{k-2} - 18x_{k-1} + 11x_k)/6T_s. \quad (45)$$

Therefore, instead of calculating ω_n , K_w is replaced in the implementation. From (34), we have

$$K_w = \begin{cases} K_1 T_s |x_e|, & \text{if } x_e \frac{\partial x_e}{\partial t} < 0 \\ K_1 T_s |x_e| + \frac{K_2}{6} |num|, & \text{if } x_e \frac{\partial x_e}{\partial t} \geq 0 \end{cases} \quad (46)$$

with $num = -2x_{ek-3} + 9x_{ek-2} - 18x_{ek-1} + 11x_{ek}$.

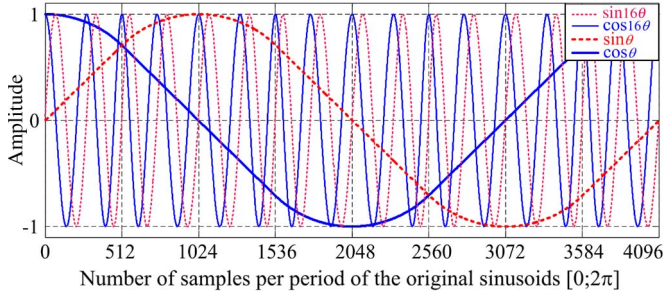


Fig. 11. Building LUTs of the 16th-order sinusoids based on the original signals.

The factors K_1 and K_2 are determined according to the experimental results. In fact, T_s is very small (about 10 μ s); however, the parameter K_1 has a large enough value to ensure the magnitude of K_w in the range of the DSP's calculation ability. We finally get a set of equations (42)–(44) and (46) which can reduce the computation burden and make possible the implementation of the low-cost fixed-point DSPs.

IV. HIGH-RESOLUTION PULSE GENERATOR

In the previous approach [22], the authors presented an upgraded method, which is based on the basic idea of Tan and Zhou [9], to increase the resolution in each period of the encoder signals. In there, the high-order sinusoids were extracted from the original signals and converted to binary pulses using the hysteresis technique. The order of extracted sinusoids is the interpolation factor N of this method. However, by many practical experiments, we realized that the higher the interpolation factor is, the larger is the noise effect on the output pulses. Therefore, undesired outputs (e.g., dither phenomena) are easily generated in this case.

One solution to resolve this problem is to reduce the order of interpolated sinusoids per period of the original signals; thus, noises will be reduced significantly. However, how can we get the same high resolution as before? This paper proposes a new scheme to achieve higher resolution from the lower order of the interpolated sinusoids. There are two main differences of this scheme compared with the previous one: first, the extraction of two high-order sinusoids that are dephased by an angle $\pi/4$ instead of only one as before, and second, the use of an additional small-size FPGA to interpolate higher pulses as well as to reduce the calculation burden of the DSP. In order to clearly understand this, we describe this scheme step by step as follows:

1) *Step 1:* After correcting the encoder signals by using the AADPLL, we make n th-order sinusoids using the same LUT method as with the previous research in [22] (see Fig. 11). However, aside from taking $\sin n\hat{\varphi}$ and $\cos n\hat{\varphi}$, we get two more signals $\sin(n\hat{\varphi} + \pi/4)$ and $\cos(n\hat{\varphi} + \pi/4)$. If M is the number of samples of LUTs and n is the order of the n th-order sinusoids, then the indexes to look up in the LUTs can be given by

$$\begin{cases} \text{index}_1 = \text{round}\left(\frac{M\hat{\varphi}}{2\pi}\right) \\ \text{index}_2 = \text{index}_1 + \text{round}\left(\frac{M}{8n}\right). \end{cases} \quad (47)$$

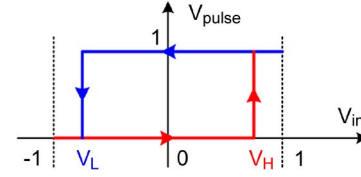


Fig. 12. Hysteresis technique to detect signal levels.

To ensure that these indexes are in the range of LUTs, a further condition should be checked:

if ($\text{index}_i > M$) then $\text{index}_i = \text{index}_i - M$, $i = 1 \div 2$.

2) *Step 2:* In practice, the n th-order sinusoids are not smooth because of the noise effect. In the conversion scheme of these signals to binary pulses, if the zero-crossing method is used to detect the signal levels, the output will fluctuate due to noises. Therefore, the hysteresis technique is applied to do this conversion. Fig. 12 shows the principle of this method. Here, two threshold-voltage levels V_L and V_H are used instead of zero level. It can be described by the two following states S_1 and S_2 :

$$\begin{cases} S_1 : u_{\text{in}} \text{ rising and } u_{\text{in}} \geq V_H \rightarrow V_{\text{out}} = 1 \text{ and hold until } S_1 \\ S_2 : u_{\text{in}} \text{ falling and } u_{\text{in}} \leq V_L \rightarrow V_{\text{out}} = 0 \text{ and hold until } S_2. \end{cases}$$

Finally, we can get two pairs of quadrature pulses (Pulse A_1 , Pulse B_1) and (Pulse A_2 , Pulse B_2) corresponding to $(\sin n\hat{\varphi}, \cos n\hat{\varphi})$ and $(\sin(n\hat{\varphi} + \pi/4), \cos(n\hat{\varphi} + \pi/4))$. After this, these pulses will be fed to the four inputs of the FPGA for extra interpolation, which is described in next step.

3) *Step 3:* This step will be done in the FPGA. From the quadrature pulses (Pulse A_1 , Pulse B_1), we can easily make a new signal with double frequency Pulse AB_1 by applying the XOR operator: Pulse AB_1 = Pulse A_1 XOR Pulse B_1 . Similarly, we can obtain Pulse AB_2 from (Pulse A_2 , Pulse B_2): Pulse AB_2 = Pulse A_2 XOR Pulse B_2 . As easily seen, the signals Pulse AB_1 and Pulse AB_2 have the same frequency f_{AB} and are equal to two times f_A (or f_B): $f_{AB} = 2f_A = 2n f_{\text{origin}}$; f_{origin} is the frequency of the encoder signals. Additionally, let us take an analysis on the phase of these pulses; we have

$$\begin{cases} \theta_{A1} = \theta_{A2} + \frac{\pi}{4} \\ \theta_{AB1} = 2\theta_{A1} \\ \theta_{AB2} = 2\theta_{A2}. \end{cases} \implies \theta_{AB1} = \theta_{AB2} + \frac{\pi}{2} \quad (48)$$

The result of (48) means that Pulse AB_1 and Pulse AB_2 are also a pair of quadrature pulses. They can be used as final outputs of this pulse generator. The output frequency is clearly increased $2n$ times from the original signals of the encoder. However, in order to analyze the resolution of the pulse generator, we apply the multiply-by-four technique (a basic technique) to get four times higher position than that of the previous one. According to this algorithm, there are two outputs: The first is a signal showing the moving direction, and the second is a pulse with double frequency (PulseOut = Pulse AB_1 XOR Pulse AB_2), which can get a position that is four times bigger by applying the edge-detection method. As a result, we totally achieve an interpolation factor that is equal to $8n$ times with respect to the original signals.

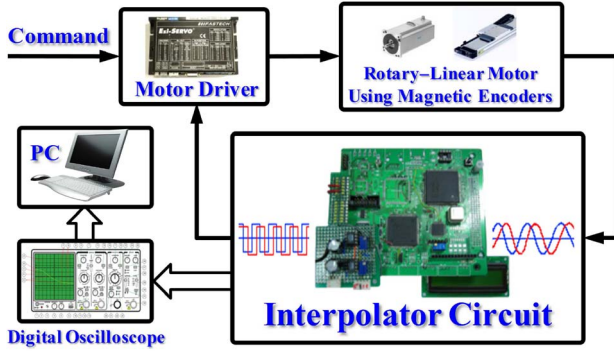


Fig. 13. System setup for testing the proposed method.

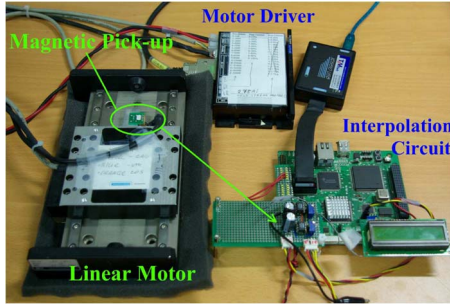


Fig. 14. Application setup: Linear-motor control with ME.

In comparison with the previous approach [22], at the same output resolution, the order of sinusoids extracted from the original ones is reduced by half in this paper. For example, with an interpolation factor $N = 128$, this method needs only $n = 16$ (16th-order sinusoids to be extracted), whereas the previous one requires a bigger order $n = 32$. Hence, the noise effect are significantly reduced from the output pulses.

V. SIMULATION AND EXPERIMENTAL RESULTS

This section will provide some simulation and experimental results to evaluate the performance of the proposed approach. Currently, all of the algorithms are implemented in a DSP-based platform, which use a low-cost DSP TMS320F2812 with 150 Mhz maximum millions of instructions per second and an FPGA Xilinx XC2S200 (actually, we can use a smaller size FPGA such as the XC2S30). A test system has been built for these experiments, as shown in Fig. 13. In this setup, the EZI-SERVO motor driver (Fastech Company) is used to control the motors (rotary/linear motors) which are mounted with the ME. Moreover, the interpolation circuit based on the aforementioned DSP/FPGA-based platform processes the encoder's signals and provides position information for the motor driver. A digital oscilloscope is connected to record data and save to a computer for easy debugging and redrawing. Some results are redrawn by the stored data in an external memory of the hardware platform. In these simulations and experiments, the parameters are chosen as follows:

- 1) For the AADPLL:
 - a) $\omega_0 = 500 \cdot 2\pi$, $K_1 = 38\,000$, and $K_2 = 0.5$;
 - b) $\omega_{n\min} = 10\,000 \cdot 2\pi$ and $\omega_{n\max} = 1000 \cdot 2\pi$;
 - c) $K_{ca} \in [5, 17]$ and $K_{cb} \in [3, 5]$ ($K_{ca} = 7$ and $K_{cb} = 3$).

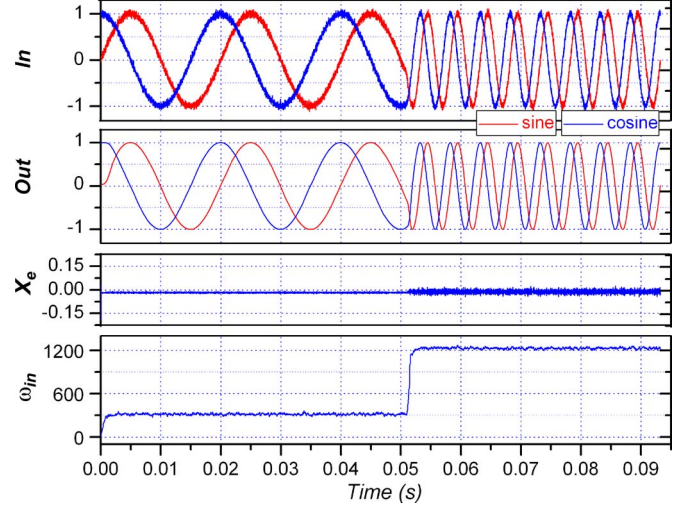


Fig. 15. Transient response of the AADPLL.

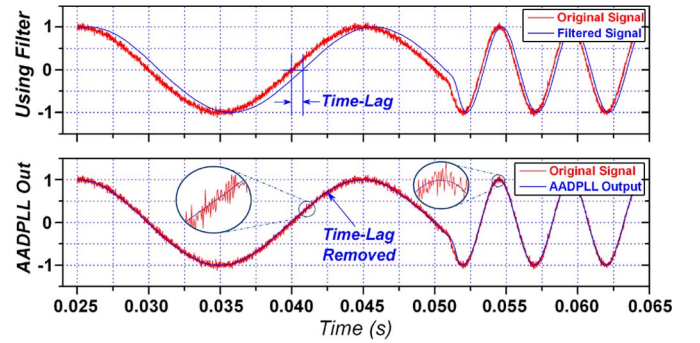


Fig. 16. Compensating outputs from time-lag effect caused by LPFs.

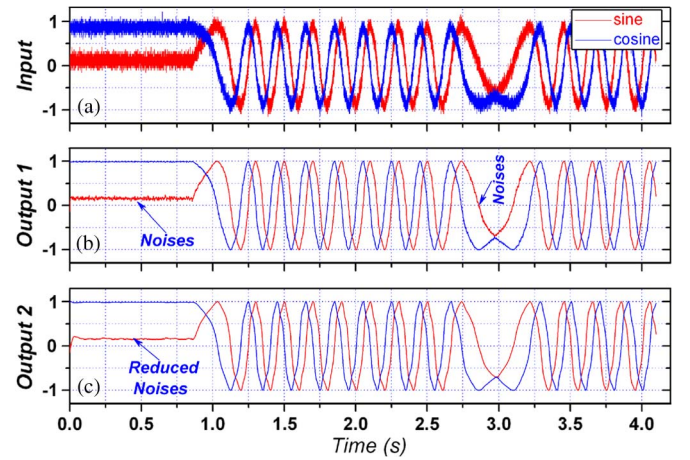


Fig. 17. (a) Input signals. (b)/(c) Output signals with/without adjusting cut-off frequencies for LPFs.

- 2) For the pulse generator:
 - a) LUTs: $M = 4096$ samples;
 - b) Interpolation factor: $N = 64, 128$, and 256 ($n = 8, 16$, and 32).

Fig. 14 shows a practical application for controlling a linear motor without using an expensive linear encoder. In this model, a quadrature magnetic pick-up GEL-2442 is mounted on the moving part in parallel with the linear motor's body. When the

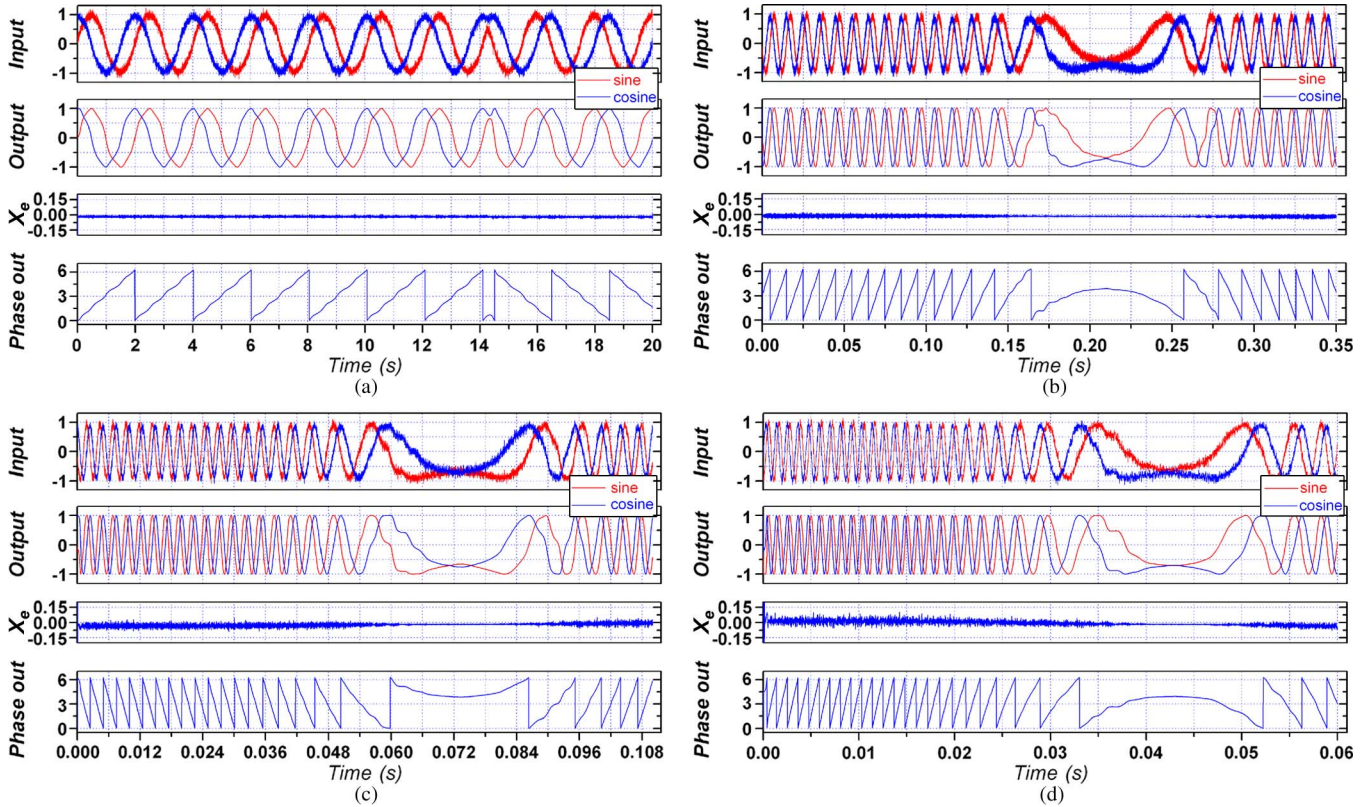


Fig. 18. Experimental results of the AADPLL applied in linear-motor control. (a) Speed of 0.5 pitch/s (~ 0.5 Hz). (b) Speed of 100 pitches/s (~ 100 Hz). (c) Speed of 400 pitches/s (~ 400 Hz). (d) Speed of 900 pitches/s (~ 900 Hz). (1 pitch ~ 1 mm).

motor runs, the magnetic pick-up slides over the fixed body of the motor, which contains a lot of pitches, and it will then generate a pair of quadrature sinusoids corresponding to each pitch. From these sinusoidal signals, we can extract the position information by using this approach. A precise optical linear encoder, Heidenhain LIDA27, was attached to the side of the linear motor to check the performance accuracy.

Fig. 15 shows the response time of the AADPLL. The input frequency was changed from 50 to 200 Hz in about 0.1 ms.

Fig. 16 shows the results of eliminating the time lag caused by the LPFs, which is applied in the APD. We can see that the AADPLL can effectively compensate this time lag and return a clear sinusoidal output.

Fig. 17 shows the effectiveness of adjusting the LPFs's cutoff frequencies in accordance with the input-signal frequency. The noises affecting the output signals become bigger in the low-frequency domains without using the ACF scheme, whereas they are significantly reduced by applying this scheme, as shown in Fig. 17(c).

The experimental results for several cases of input signals are shown in Fig. 18. Four input frequencies were tested: 0.5, 100, 400, and 900 Hz, corresponding to the linear motor speeds: 0.5, 100, 400, and 900 mm/s. The phaseout is also shown in these figures. We can realize that the AADPLL can track the signals in both very low and high frequency domains. Moreover, although the encoder signals become rough during the change of direction of rotation, the AADPLL still provides a good tracking performance in this situation.

Fig. 19 shows the simulation results of the proposed pulse generator. The practical results are shown in Fig. 20. In there,

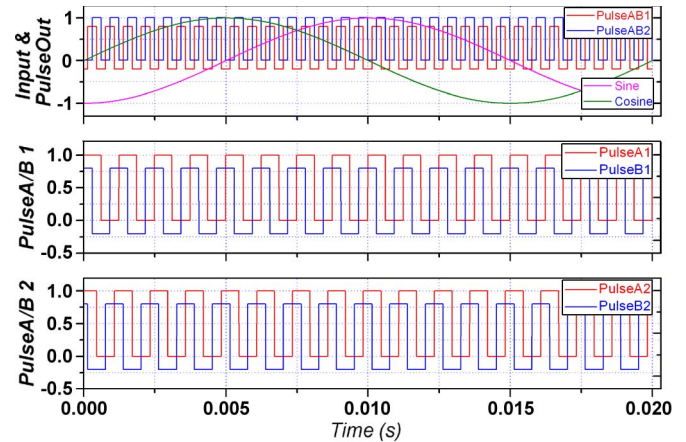


Fig. 19. Simulation results of the pulse generator ($n = 16$ and $N = 128$).

the 16th-order sinusoids are extracted from the original one and then converted to pulses, PulseA/B1 and PulseA/B2, which are dephased at an angle of $\pi/4$ from each other (more details in Section IV). The FPGA will change these pulses to a final pair of outputs, PulseAB1 and PulseAB2. The number of positions that can be counted in one period in this case is $N = 128$. Fig. 20(b) shows the interpolation results at the reversal of the direction of motion. Although the encoder signals are rough, the pulse generator still provides the correct pulses.

Table I shows the comparison of the proposed approach with some of the previous approaches that we referred to in this paper.

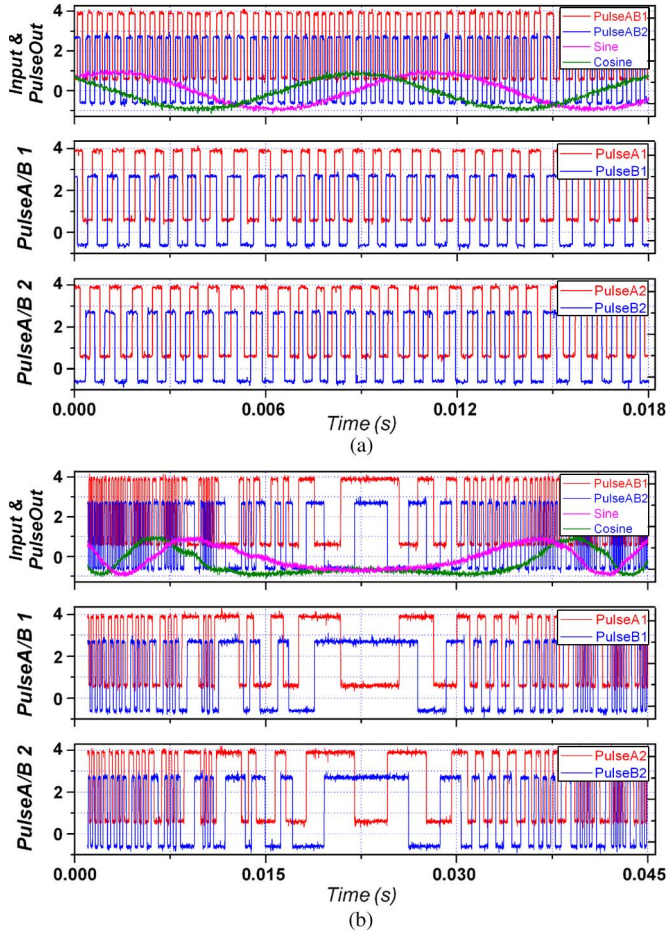


Fig. 20. Experimental results of the pulse generator. (a) $n = 16$ pulses/period, Interpolation factor $N = 128$. (b) Pulse generation at the reversed direction of motor.

To compare the maximum frequency of the output pulses between this approach and the previous one by the same authors [22], we assume that the order of the n th-order sinusoids extracted from the ME's signals is the same. In the previous approach, these n th-order sinusoids are exactly converted to the final output pulses. Therefore, the maximum output frequency is limited by $1/4T_s$ (detailed in [22]). In this approach, however, these signals continue to be interpolated twice by using an extra FPGA circuit; thus, the maximum output frequency is increased twice. It means that

$$\text{Maximum output frequency} = 2 \cdot \frac{1}{4T_s} = \frac{1}{2T_s}. \quad (49)$$

In this paper, we have already experimented with $n = 8, 16$, and 32 (corresponding to interpolation factors $N = 64, 128$, and 256). However, the higher the value of interpolation factor N is, the lower becomes the value of the maximum speed of the motor. This is true in a real motor-control system. For example, with $N = 64$, the maximum speed a linear motor can be run correctly is about 900 mm/s (with linear motor having 1 mm/pitch), and with $N = 256$, this speed is reduced to about 220 mm/s.

TABLE I
COMPARISON OF TECHNIQUES

Techniques	[15]	[9]	[17]	Proposed
	Hardware	LUTs	Quadrature	AADPLL &
Characteristics	- based		PLL	Interpolation
Real-time	✓	—	✓	✓
Compensation				
Noises (Phaseshift,				
Offset, Distortion,	—	✓	✓	✓
...) Removal				
Noises Removal	—	—	—	✓
in low frequencies				
Time-lag	—	—	—	✓
Removal				
Higher	—	✓	—	✓
Interpolation				

"—": No information about this factor is given in the reference.

VI. CONCLUSION

In this paper, the authors have proposed a full design to process the signals in real time for MEs, in particular, and sinusoidal encoders, in general. Two main objectives were achieved: 1) compensating the encoder signals for the noises and 2) interpolating to higher resolution from the aforementioned compensated waves. The first algorithm, which is called the AADPLL, was introduced to adaptively track and correct the encoder signals from noises, dc offset, phase shift, and waveform distortion. This method has the advantages of removing the time lag caused by LPFs, changing the cutoff frequency of the applied filters and adjusting the AADPLL parameters to suitable values in real time as well. The second scheme employed in the pulse generator has the ability to increase the resolution for the encoders, which is represented by the interpolation factor N ($N = 8n$, where n is the order of the high-order sinusoids extracted from the original encoder signals). The combination of the low-cost DSP and the small-size FPGA is a nice idea to improve the quality of the output pulses.

The feasibility of the proposed method has been verified by the simulation and experimental results. Applying this approach, the linear-motor-control systems can reduced significantly the price by replacing an expensive optical encoder with a simple-structure ME. It is easily made by mounting a quadrature magnetic pickup (e.g., GEL-2442) on the moving part in parallel with the linear motor's body. In the future, it is feasible to implement all of the algorithms in an FPGA to increase the system's performance and to manufacture an application-specific integrated circuit using sinusoidal encoders in general.

REFERENCES

- [1] P. L. M. Heydemann, "Determination and correction of quadrature fringe measurement errors in interferometers," *Appl. Opt.*, vol. 20, no. 19, pp. 3382–3384, Oct. 1981.
- [2] S. Balemi, "Automatic calibration of sinusoidal encoder signals," in *Proc. 16th IFAC World Congr.*, Prague, Czech Republic, Jul. 2005.

- [3] H. V. Hoang and J. W. Jeon, "Signal compensation and extraction of high-resolution position for sinusoidal magnetic encoders," in *Proc. ICASS*, Oct. 2007, pp. 1368–1373.
- [4] R. Hoseinnezhad, A. Bab-Hadiashar, and P. Harding, "Calibration of resolver sensors in electromechanical braking systems: A modified recursive weighted least-squares approach," *IEEE Trans. Ind. Electron.*, vol. 54, no. 2, pp. 1052–1060, Apr. 2007.
- [5] N. Hagiwara and H. Murase, "A method of improving the resolution and accuracy of rotary encoders using a code compensation technique," *IEEE Trans. Instrum. Meas.*, vol. 41, no. 1, pp. 98–101, Feb. 1992.
- [6] T. Kiriya and M. Teraguchi, "Interpolation circuit for encoder having a look-up table memory with reduced capacity," U.S. patent 5999 113, Dec. 7, 1999.
- [7] C. Attianese and G. Tomasso, "Position measurement in industrial drives by means of low-cost resolver-to-digital converter," *IEEE Trans. Instrum. Meas.*, vol. 56, no. 6, pp. 2155–2159, Dec. 2007.
- [8] S. Sarma, V. K. Agrawal, and S. Udupa, "Software-based resolver-to-digital conversion using a DSP," *IEEE Trans. Ind. Electron.*, vol. 55, no. 1, pp. 371–379, Jan. 2008.
- [9] K. K. Tan and H. X. Zhou, "New interpolation method for quadrature encoder signals," *IEEE Trans. Instrum. Meas.*, vol. 51, no. 5, pp. 1073–1079, Oct. 2002.
- [10] K. K. Tan and K. Z. Tang, "Adaptive online correction and interpolation of quadrature encoder signals using radial basis functions," *IEEE Trans. Control Syst. Technol.*, vol. 13, no. 3, pp. 370–377, May 2005.
- [11] R. C. Kavanagh, "Probabilistic learning technique for improved accuracy of sinusoidal encoders," *IEEE Trans. Ind. Electron.*, vol. 48, no. 3, pp. 673–681, Jun. 2001.
- [12] A. Bunte and S. Beineke, "High-performance speed measurement by suppression of systematic resolver and encoder errors," *IEEE Trans. Ind. Electron.*, vol. 51, no. 1, pp. 49–53, Feb. 2004.
- [13] M. Benammar, "A novel amplitude-to-phase converter for sine/cosine position transducers," *Int. J. Electron.*, vol. 94, no. 4, pp. 353–365, Apr. 2007.
- [14] L. Ben-Brahim, M. Benammar, M. A. Alhamadi, N. A. Al-Emadi, and M. A. Al-Hitmi, "A new low cost linear resolver converter," *IEEE Sensors J.*, vol. 8, no. 10, pp. 1620–1627, Oct. 2008.
- [15] L. Ben-Brahim, M. Benammar, and M. A. Alhamadi, "Estimation based on its excitation signal," *IEEE Trans. Ind. Electron.*, vol. 56, no. 2, pp. 574–580, Feb. 2009.
- [16] A. Michalski, J. Sienkiewicz, and Z. Watral, "Universal magnetic circuit for resolvers with different speed ratios," *IEEE Instrum. Meas. Mag.*, vol. 10, no. 5, pp. 58–68, Oct. 2007.
- [17] T. Emura and L. Wang, "A high-resolution interpolation for incremental encoders based on the quadrature PLL method," *IEEE Trans. Ind. Electron.*, vol. 47, no. 1, pp. 84–90, Feb. 2000.
- [18] A. Bellini and S. Bifaretti, "A digital filter for speed noise reduction in drives using an electromagnetic resolver," *Math. Comput. Simul.*, vol. 71, no. 4–6, pp. 476–486, Jun. 2006.
- [19] J. O. Krah, H. Schmirgel, and M. Albers, "FPGA based resolver to digital converter using delta-sigma technology," in *Proc. Power Electron. Intell. Motion, Power Quality Europe*, 2006, pp. 931–936.
- [20] L. Idkhajine, E. Monmasson, M. W. Naoouar, A. Prata, and K. Bouallaga, "Fully integrated FPGA-based controller for synchronous motor drive," *IEEE Trans. Ind. Electron.*, vol. 56, no. 10, pp. 4006–4017, Oct. 2009.
- [21] S. Sayeef, G. Foo, and M. F. Rahman, "Rotor position and speed estimation of a variable structure direct-torque-controlled IPM synchronous motor drive at very low speeds including standstill," *IEEE Trans. Ind. Electron.*, vol. 57, no. 11, pp. 3715–3723, Nov. 2010.
- [22] H. V. Hoang, H. T. Le, and J. W. Jeon, "A new approach based on advanced adaptive digital PLL for improving the resolution and accuracy of magnetic encoders," in *Proc. IEEE/RSJ Int. Conf. IROS*, Sep. 2008, pp. 3318–3323.
- [23] H. T. Le, H. V. Hoang, and J. W. Jeon, "Efficient method for correction and interpolation signal of magnetic encoders," in *Proc. IEEE Int. Conf. INDIN*, Jul. 2008, pp. 1383–1388.
- [24] I. Godler, "A method to compensate periodic errors by gain tuning in instrumentation," *IEEE Trans. Instrum. Meas.*, vol. 51, no. 1, pp. 37–42, Feb. 2002.
- [25] J. Roche, W. Rahajandraibe, L. Zad, and G. Bracmard, "A new adaptation scheme for low noise and fast settling phase locked loop," in *Proc. 51st MWSCAS*, Aug. 2008, pp. 197–200.
- [26] C. S. Vaucher, "An adaptive PLL tuning system architecture combining high spectral purity and fast settling time," *IEEE J. Solid-State Circuits*, vol. 35, no. 4, pp. 490–502, Apr. 2000.
- [27] J. Lee and B. Kim, "A low-noise fast-lock phase-locked loop with adaptive bandwidth control," *IEEE J. Solid-State Circuits*, vol. 35, no. 8, pp. 1137–1145, Aug. 2000.
- [28] T. Tsuji, T. Hashimoto, H. Kobayashi, M. Mizuochi, and K. Ohnishi, "A wide-range velocity measurement method for motion control," *IEEE Trans. Ind. Electron.*, vol. 56, no. 2, pp. 510–519, Feb. 2009.
- [29] W. Zhu and T. Lamarche, "Velocity estimation by using position and acceleration sensors," *IEEE Trans. Ind. Electron.*, vol. 54, no. 5, pp. 2706–2715, Oct. 2007.
- [30] R. E. Best, *Phase-Locked Loops: Design, Simulation, and Applications*, 5th ed. New York: McGraw-Hill, 2003.
- [31] F. M. Gardner, *Phaselock Techniques*, 3rd ed. Hoboken, NJ: Wiley, 2005.
- [32] L. D. Paarmann, *Design and Analysis of Analog Filters: A Signal Processing Perspective*, 3rd ed. Boston, MA: Kluwer, 2001.
- [33] J. D. Hoffman, *Numerical Methods for Engineering and Scientists*, 2nd ed. New York: McGraw-Hill, 2001.



Hung Van Hoang received the B.S. degree in electrical and electronics engineering from the Ho Chi Minh City University of Technology, Ho Chi Minh City, Vietnam, in 2003 and the M.S. degree in electrical and computer engineering from Sungkyunkwan University, Suwon, Korea, in 2008, where he is currently working toward the Ph.D. degree in the School of Information and Communications Engineering.

His research interests include motion control, embedded systems, and real-time applications.



Jae Wook Jeon (S'82–M'84) received the B.S. and M.S. degrees in electronics engineering from Seoul National University, Seoul, Korea, in 1984 and 1986, respectively, and the Ph.D. degree in electrical engineering from Purdue University, West Lafayette, IN, in 1990.

From 1990 to 1994, he was a Senior Researcher with Samsung Electronics, Suwon, Korea. Since 1994, he has been with Sungkyunkwan University, Suwon, where he was first an Assistant Professor with the School of Electrical and Computer Engineering and is currently a Professor with the School of Information and Communication Engineering. His research interests include robotics, embedded systems, and factory automation.